REVISION CHEAT SHEET:

Generics in C#

TOPIC 20 · C#

Interview-ready notes · examples · C# · diagrams

Topic 20 of 40

Generics allow you to write:

EXAMPLE / FLOW

```
One implementation
    ↓
Works with many types
    ↓
While preserving compile-time type safety
```

Instead of creating:

C#

```
IntRepository

StringRepository

GiftRepository

CampaignRepository

FundRepository
```

you can create a reusable type:

C#

```
Repository<T>
```

and then use:

C#

```
Repository<Gift>

Repository<Campaign>

Repository<Fund>
```

The type:

EXAMPLE / FLOW

```
T
```

acts as a placeholder.

When the generic type is used, the caller supplies the actual type.

Microsoft defines generics as type-parameterized classes, structures, interfaces, and methods where type parameters act as placeholders for the actual types supplied by consumers. Their main benefits include type safety, code reuse, and—in many value-type scenarios—avoiding boxing.

High-Level Mental Model

Without generics:

EXAMPLE / FLOW

```
Gift storage
    →
Gift-specific code

Campaign storage
    →
Campaign-specific code

Fund storage
    →
Fund-specific code
```

With generics:

EXAMPLE / FLOW

```
Repository<T>
        ↓
  ┌─────┼─────┐
  ↓     ↓     ↓
Gift Campaign  Fund
```

One reusable design.

Different concrete type arguments.

Core Syntax

C#

```
public class Box<T>
{
    public T Value
    {
        get;
        set;
    }
}
```

Usage:

C#

```
Box<int> numberBox = new();

numberBox.Value = 100;
```

Another constructed type:

C#

```
Box<string> textBox = new();

textBox.Value = "Annual Giving";
```

Same generic definition:

EXAMPLE / FLOW

```
Box<T>
```

TOPIC 20 · C#

Two different constructed types:

EXAMPLE / FLOW

```
Box<int>

Box<string>
```

Why Generics Exist

Before generics, reusable APIs often relied heavily on:

C#

```
object
```

Example:

C#

```
public sealed class OldBox
{
    public object? Value
    {
        get;
        set;
    }
}
```

Usage:

C#

```
OldBox box = new();

box.Value = 100;
```

Later:

C#

```
int value = (int)box.Value;
```

Problems include:

EXAMPLE / FLOW

```
Runtime casting

Potential InvalidCastException

Loss of compile-time type information

Boxing for value types

Less readable APIs
```

Generic Version

```C#
public sealed class Box<T>
{
    public T? Value
    {
        get;
        set;
    }
}
```

Usage:

```C#
Box<int> box = new();

box.Value = 100;
```

The compiler knows:

EXAMPLE / FLOW

```
T = int
```

so:

```C#
int value = box.Value;
```

doesn't require a runtime cast.

Generics move much of the type-safety work to compile time and reduce the need for explicit casts.

Generic Terminology

Understanding the terminology is important for interviews and documentation.

Consider:

```C#
public class Repository<T>
{
}
```

Generic Type Definition

EXAMPLE / FLOW

```
Repository<T>
```

is the generic definition.

Think:

EXAMPLE / FLOW

```
Blueprint
```

The type parameter hasn't yet been replaced by a concrete type.

Type Parameter

In:

```C#
Repository<T>
```

`T` is a:

```EXAMPLE / FLOW
Type Parameter
```

It represents a type that will be supplied later.

Type Argument

When you write:

```C#
Repository<Gift>
```

`Gift` is the:

```EXAMPLE / FLOW
Type Argument
```

It replaces:

```EXAMPLE / FLOW
T
```

for that constructed type.

Constructed Generic Type

```C#
Repository<Gift>
```

is a constructed generic type.

Microsoft uses the term **constructed type** for a generic definition after type arguments have been supplied.

Simple Substitution Mental Model

Definition:

```C#
public class Repository<T>
{
    public T? Find()
    {
        return default;
    }
}
```

Usage:

```
C#
Repository<Gift>
```

Conceptually behaves as though:

```
EXAMPLE / FLOW
T
    ↓
Gift
```

so:

```
C#
T? Find()
```

becomes conceptually:

```
C#
Gift? Find()
```

Generic Type Parameters Are Types

Inside:

```
C#
public class Holder<T>
{
}
```

`T` can be used wherever the language permits the corresponding type parameter.

For example:

```
C#
private T _value;

public T Value
{
    get;
    set;
}

public T GetValue()
{
    return _value;
}

public void SetValue(
    T value)
{
    _value = value;
}
```

Naming Generic Parameters

Simple single-type generic code often uses:

Example:

```C#
List<T>
```

For multiple meaningful parameters, use descriptive names.

Examples:

```EXAMPLE / FLOW
TKey

TValue

TEntity

TResult

TRequest

TResponse

TSource

TDestination
```

Example:

```C#
public interface IRepository<TEntity>
{
}
```

is often clearer than:

```C#
public interface IRepository<T>
{
}
```

when the domain meaning matters.

Multiple Generic Parameters

A generic type can have more than one type parameter.

```C#
public sealed class Pair<TKey, TValue>
{
    public Pair(
        TKey key,
        TValue value)
    {
        Key = key;

        Value = value;
    }
```

```
    public TKey Key
    {
        get;
    }

    public TValue Value
    {
        get;
    }
}
```

Usage:

C#
```
Pair<long, Gift> pair = new(1001, gift);
```

Here:

EXAMPLE / FLOW
```
TKey
    →
long

TValue
    →
Gift
```

Real .NET Example — Dictionary

You already use this idea constantly:

C#
```
Dictionary<long, Gift>
```

where:

EXAMPLE / FLOW
```
TKey
    →
long

TValue
    →
Gift
```

The generic definition is:

EXAMPLE / FLOW
```
Dictionary<TKey, TValue>
```

Generics Are Everywhere in .NET

Examples include:

C#
```
List<Gift>

Dictionary<long, Gift>
```

```
Task<Gift>

IEnumerable<Gift>

IReadOnlyList<Gift>

Func<Gift, bool>

Nullable<int>
```

Generics underpin many of the most commonly used .NET APIs.

Generic Classes

A class can have type parameters.

C#
```
public sealed class Repository<TEntity>
{
    private readonly List<TEntity>
        _items = [];

    public void Add(
        TEntity item)
    {
        _items.Add(
            item);
    }

    public IReadOnlyList<TEntity>
        GetAll()
    {
        return _items;
    }
}
```

Use it for Gifts:

C#
```
Repository<Gift> gifts = new();
```

Campaigns:

C#
```
Repository<Campaign> campaigns = new();
```

Funds:

C#
```
Repository<Fund> funds = new();
```

Type Safety

Once declared:

C#
```
Repository<Gift> repository = new();
```

this is valid:

```C#
repository.Add(
    gift);
```

This is not:

```C#
// repository.Add(
//     campaign);
```

The compiler knows the repository accepts:

EXAMPLE / FLOW

```
Gift
```

not:

EXAMPLE / FLOW

```
Campaign
```

Generic Structs

Structs may also be generic.

Example:

```C#
public readonly struct Result<T>
{
    public Result(
        T value)
    {
        Value = value;

        Success = true;
    }

    public T? Value
    {
        get;
    }

    public bool Success
    {
        get;
    }
}
```

Usage:

```C#
Result<Gift> result = new(gift);
```

Generic Records

Records can also use generic type parameters.

Example:

```C#
public sealed record ApiResponse<T>(
    T Data,
    string Message);
```

Usage:

```C#
ApiResponse<GiftResponse>
    response = new(giftResponse, "Gift retrieved.");
```

Another:

```C#
ApiResponse<IReadOnlyList<GiftResponse>>
```

This is common in API/data-oriented models, although you should avoid wrapping every API response merely because generics make it possible.

Generic Interfaces

Interfaces can also be generic.

Example:

```C#
public interface IRepository<TEntity>
{
    TEntity? Find(
        long id);

    void Add(
        TEntity entity);
}
```

Implementation:

```C#
public sealed class GiftRepository :
    IRepository<Gift>
{
    public Gift? Find(
        long id)
    {
        return null;
    }

    public void Add(
        Gift entity)
    {
    }
}
```

Generic interfaces are widely used throughout .NET and can also appear as generic constraints.

Closed Interface Implementation

This:

```C#
public sealed class GiftRepository :
    IRepository<Gift>
```

means:

EXAMPLE / FLOW

```
IRepository<TEntity>
    ↓
TEntity = Gift
```

So the implementation must satisfy:

```C#
Gift? Find(
    long id);

void Add(
    Gift entity);
```

Generic Implementation

A class itself can remain generic:

```C#
public sealed class Repository<TEntity> :
    IRepository<TEntity>
{
    public TEntity? Find(
        long id)
    {
        return default;
    }

    public void Add(
        TEntity entity)
    {
    }
}
```

Now:

```C#
Repository<Gift>
```

implements:

```C#
IRepository<Gift>
```

while:

```C#
Repository<Campaign>
```

implements:

```
C#
IRepository<Campaign>
```


Generic Methods

A method can have its own generic type parameters.

Example:

```
C#
public static T Echo<T>(
    T value)
{
    return value;
}
```

Calls:

```
C#
int number = Echo<int>(10);

string text = Echo<string>("Gift");
```

Generic Method Type Inference

Usually the compiler can infer the type.

Instead of:

```
C#
int number = Echo<int>(10);
```

you can write:

```
C#
int number = Echo(10);
```

The compiler sees:

```
EXAMPLE / FLOW
Argument type
    =
int
```

and infers:

```
EXAMPLE / FLOW
T
    =
int
```

Another Example

```
C#
public static T First<T>(
    T first,
    T second)
```

```
{
    return first;
}
```


Call:

```C#
string value = First("Gift", "Campaign");
```

The compiler determines:

EXAMPLE / FLOW

```
T = string
```

Type Inference Uses Arguments, Not Return Assignment Alone

Consider:

```C#
public static T Create<T>()
{
    return default!;
}
```

This normally requires:

```C#
Gift gift = Create<Gift>();
```

You generally cannot rely solely on:

```C#
Gift gift = Create();
```

to infer `T`, because generic method inference primarily works from method arguments rather than the assignment target.

A Method Inside a Generic Class Is Not Necessarily Generic

Consider:

```C#
public class Repository<TEntity>
{
    public TEntity Get()
    {
        return default!;
    }
}
```

`Repository<TEntity>` is generic.

But:

```C#
Get()
```

doesn't declare its own type parameter.

So `Get` itself isn't a generic method.

Microsoft explicitly distinguishes a generic method from a normal method that merely uses its containing generic type's type parameter.

A Generic Method Has Its Own Type Parameters

```C#
public TResult Convert<TResult>(
    TEntity entity)
{
    return default!;
}
```

Now:

EXAMPLE / FLOW

```
<TEntity>
```

belongs to the class.

EXAMPLE / FLOW

```
<TResult>
```

belongs to the method.

Example

```C#
public sealed class Mapper<TSource>
{
    public TResult Map<TResult>(
        TSource source)
    {
        return default!;
    }
}
```

Usage:

```C#
Mapper<Gift> mapper = new();

GiftResponse response = mapper.Map<GiftResponse>(gift);
```

Why Constraints Exist

Suppose:

```C#
public static void PrintName<T>(
    T value)
{
    Console.WriteLine(
        value.Name);
}
```

This does not compile.

Why?

The compiler only knows:

EXAMPLE / FLOW

```
T
could theoretically be
anything.
```

`int` doesn't have your domain `Name`.

`DateTime` doesn't necessarily have it.

`Gift` may.

`Campaign` may.

But unconstrained `T` makes no promise.

Solution — Generic Constraints

You tell the compiler:

EXAMPLE / FLOW

```
T must satisfy these requirements.
```

Example:

C#

```
public interface INamedEntity
{
    string Name
    {
        get;
    }
}
```

Then:

C#

```
public static void PrintName<T>(
    T value)
    where T :
        INamedEntity
{
    Console.WriteLine(
        value.Name);
}
```

Now the compiler knows every valid `T` has:

C#

```
Name
```

Constraints restrict the allowed type arguments and enable the generic implementation to use members guaranteed by those constraints.

where Constraint Syntax

C#

```
public class Repository<TEntity>
    where TEntity :
        class
{
}
```

General pattern:

EXAMPLE / FLOW

```
Generic declaration
    ↓
where TypeParameter : Constraint
```

Major Constraint Categories

Modern C# supports constraints such as:

EXAMPLE / FLOW

```
where T : class

where T : class?

where T : struct

where T : notnull

where T : unmanaged

where T : BaseClass

where T : IInterface

where T : U

where T : new()

where T : default

where T : allows ref struct
```

Some constraints cannot be combined, and several have required ordering rules.

where T : class

Example:

C#

```
public sealed class ReferenceCache<T>
    where T :
        class
{
}
```

This restricts `T` to:

EXAMPLE / FLOW

Reference types

TOPIC 20 · C#

Examples:

```C#
ReferenceCache<Gift>

ReferenceCache<string>
```

Not:

```C#
// ReferenceCache<int>
```

Nullable Reference Types and `class`

With nullable-reference analysis enabled:

```C#
where T : class
```

expresses a non-nullable reference-type constraint.

```C#
where T : class?
```

allows nullable or non-nullable reference types.

The precise nullability diagnostics depend on the consuming nullable context.

`where T : struct`

Example:

```C#
public static T Zero<T>()
    where T :
        struct
{
    return default;
}
```

This restricts:

EXAMPLE / FLOW

T

to non-nullable value types.

Examples:

EXAMPLE / FLOW

```
int

decimal

DateTime
```

```
GiftId
```


Important: `struct` Excludes Nullable

This is not allowed:

```C#
// SomeGeneric<int?>
```

when:

```C#
where T : struct
```

because `struct` means a **non-nullable value type** constraint.

`struct` and `new()`

Don't write:

```C#
// where T : struct, new()
```

The `struct` constraint already guarantees value-type construction semantics, and C# doesn't allow combining `struct` with `new()`.

`where T : notnull`

Example:

```C#
public sealed class EntityLookup<TKey, TValue>
    where TKey :
        notnull
{
}
```

This means the key is intended to be:

```
EXAMPLE / FLOW
Non-nullable
```

whether the type is:

```
EXAMPLE / FLOW
Reference type
```

or:

```
EXAMPLE / FLOW
Value type
```

This is particularly useful for dictionary-like keys.

Example

```C#
public sealed class Cache<TKey, TValue>
    where TKey :
        notnull
{
    private readonly Dictionary<TKey, TValue>
        _values = [];
}
```

`Dictionary<TKey,TValue>` itself uses a non-null key constraint in modern .NET API annotations.

`notnull` Is Not Runtime Validation

This is important.

A generic constraint primarily participates in:

```EXAMPLE / FLOW
Compile-time type checking
```

It doesn't automatically inspect arbitrary runtime values and throw if someone somehow supplies null.

Generic constraints are not a replacement for runtime input validation.

`where T : unmanaged`

Example:

```C#
public static unsafe int SizeOf<T>()
    where T :
        unmanaged
{
    return sizeof(T);
}
```

`unmanaged` means:

```EXAMPLE / FLOW
Non-nullable unmanaged value type
```

whose fields recursively contain no managed references.

Examples can include:

```EXAMPLE / FLOW
int

double

certain structs containing only unmanaged fields
```

Why `unmanaged` Exists

It is useful for:

EXAMPLE / FLOW

```
Low-level memory operations

Interop

Binary serialization

Pointers

High-performance buffers
```

It is not something typical domain services need frequently.

unmanaged Implies Value-Type Restrictions

You cannot combine:

C#

```
where T :
    unmanaged,
    new()
```

because `unmanaged` already carries the relevant value-type construction guarantee, and the language disallows that combination.

Base-Class Constraint

Example:

C#

```
public abstract class Entity
{
    public long Id
    {
        get;
        init;
    }
}
```

Then:

C#

```
public sealed class Repository<TEntity>
    where TEntity :
        Entity
{
    public long GetId(
        TEntity entity)
    {
        return entity.Id;
    }
}
```

Now compiler knows:

EXAMPLE / FLOW

```
Every TEntity
IS AN Entity
```

so:

```
C#
entity.Id
```

is legal.

Fundraising Example

```
C#
public abstract class FundraisingEntity
{
    public long TenantId
    {
        get;
        init;
    }
}
```

Then:

```
C#
public class TenantEntityProcessor<TEntity>
    where TEntity :
        FundraisingEntity
{
    public long GetTenantId(
        TEntity entity)
    {
        return entity.TenantId;
    }
}
```

Valid:

```
C#
TenantEntityProcessor<Gift>

TenantEntityProcessor<Campaign>

TenantEntityProcessor<Fund>
```

if those classes derive from `FundraisingEntity`.

Interface Constraint

Often preferable when you only need a capability.

```
C#
public interface ITenantScoped
{
    long TenantId
    {
        get;
    }
}
```

Then:

```C#
public sealed class TenantValidator<TEntity>
    where TEntity :
        ITenantScoped
{
    public bool BelongsTo(
        TEntity entity,
        long tenantId)
    {
        return entity.TenantId
            == tenantId;
    }
}
```

This doesn't force all entities into one class hierarchy.

Multiple Interface Constraints

```C#
public sealed class Processor<T>
    where T :
        ITenantScoped,
        IAuditable
{
}
```

Every valid `T` must implement:

EXAMPLE / FLOW

```
ITenantScoped
AND
IAuditable
```

Base Class + Interface Constraints

```C#
public sealed class Processor<T>
    where T :
        FundraisingEntity,
        IAuditable
{
}
```

Meaning:

EXAMPLE / FLOW

```
T must derive from FundraisingEntity

AND

T must implement IAuditable
```

new() Constraint

Example:

```C#
public static T Create<T>()
    where T :
        new()
{
    return new T();
}
```

The constraint guarantees an accessible public parameterless constructor.

Without:

```C#
where T : new()
```

this isn't generally legal:

```C#
new T();
```

Example

```C#
public class GiftDraft
{
    public GiftDraft()
    {
    }
}
```

Then:

```C#
GiftDraft draft = Create<GiftDraft>();
```

new() Doesn't Accept Every Constructor

This:

```C#
public class Gift
{
    public Gift(
        decimal amount)
    {
    }
}
```

doesn't satisfy:

```C#
where T : new()
```

because there is no accessible public parameterless constructor.

`new()` Must Come Last Among Ordinary Constraints

Example:

```C#
public sealed class Factory<T>
    where T :
        class,
        IEntity,
        new()
{
}
```

`new()` must appear after the other ordinary constraints. Anti-constraints such as `allows ref struct`, where applicable, come after the normal constraints.

Why `new()` Is Sometimes Overused

Suppose:

```C#
public class Repository<TEntity>
    where TEntity :
        class,
        new()
```

But repository never writes:

```C#
new TEntity();
```

Then:

EXAMPLE / FLOW

```
new()
```

is unnecessary.

Unnecessary constraints reduce what types consumers can use.

Only constrain what your generic implementation actually needs.

Type-Parameter Constraint

A generic parameter can depend on another generic parameter.

Example:

```C#
public static void Copy<TSource, TTarget>()
    where TTarget :
        TSource
{
}
```

This expresses a relationship between type parameters.

Another Example


```C#
public sealed class Converter<TBase, TDerived>
    where TDerived :
        TBase
{
}
```

Every valid `TDerived` must be assignable to `TBase`.

This can be useful in advanced generic API design.

Constraint Ordering

C# has strict ordering rules.

A simplified pattern is:

```EXAMPLE / FLOW
Primary constraint
    ↓
Base/interface constraints
    ↓
new()
    ↓
anti-constraint
```

For example:

```C#
where T :
    class,
    IComparable<T>,
    new()
```

Modern C# permits at most one of:

```EXAMPLE / FLOW
struct

class

class?

notnull

unmanaged
```

for a given type parameter, and if supplied it must generally appear first. `new()` must be last among ordinary constraints, while `allows ref struct` must follow the other constraints.

default(T)

For an unconstrained generic type:

```C#
public static T GetDefault<T>()
{
```

```
    return default(T);
}
```


What does that return?

Depends on `T`.

If T Is a Value Type

```C#
GetDefault<int>()
```

returns:

EXAMPLE / FLOW

```
0
```

```C#
GetDefault<bool>()
```

returns:

EXAMPLE / FLOW

```
false
```

For a struct, its zero/default state is returned.

If T Is a Reference Type

```C#
GetDefault<Gift>()
```

returns:

EXAMPLE / FLOW

```
null
```

`default` Literal

Modern C# also allows:

```C#
public static T GetDefault<T>()
{
    return default;
}
```

The compiler uses the target type:

EXAMPLE / FLOW

```
T
```

to determine the default value.

Why Generic `default` Exists

You cannot write:

```C#
return null;
```

for every possible `T` because:

EXAMPLE / FLOW

```
T might be int
```

And you cannot write:

```C#
return 0;
```

because:

EXAMPLE / FLOW

```
T might be Gift
```

But:

```C#
return default;
```

works for both.

Be Careful with Generic Default Values

Example:

```C#
public static T FindOrDefault<T>()
{
    return default;
}
```

For:

```C#
T = Gift
```

default is:

EXAMPLE / FLOW

```
null
```

For:

```C#
T = int
```

default is:

EXAMPLE / FLOW

```
0
```

TOPIC 20 · C#

But maybe:

EXAMPLE / FLOW

```
0
```

is a legitimate business value.

Therefore `default(T)` doesn't necessarily communicate:

EXAMPLE / FLOW

```
Not found
```

unambiguously.

`default` Constraint

There is also a generic constraint named:

C#

```
where T : default
```

This is **not** the same as:

C#

```
default(T)
```

The `default` constraint is a specialized language feature used primarily on:

EXAMPLE / FLOW

```
Overrides

Explicit interface implementations
```

to resolve ambiguity involving an unconstrained nullable generic type parameter.

It isn't a normal everyday constraint for ordinary application code. Microsoft restricts it to override and explicit-interface implementation scenarios.

Modern Anti-Constraint — `allows ref struct`

C# 13 introduced:

C#

```
where T :
    allows ref struct
```

This says:

> `T` is allowed to be a `ref struct` type.

Before C# 13, `ref struct` types such as `Span<T>` couldn't be supplied as generic type arguments in this way.

Example

```C#
public static void Process<T>(
    scoped T value)
    where T :
        allows ref struct
{
}
```

Now a ref-struct type can be permitted where the generic implementation obeys all applicable ref-safety rules.

Why Is It Called an Anti-Constraint?

Traditional constraint:

```EXAMPLE / FLOW
Restricts what T may be.
```

For example:

```C#
where T : class
```

says:

```EXAMPLE / FLOW
T must be a reference type.
```

`allows ref struct` instead says:

```EXAMPLE / FLOW
Do not automatically exclude
ref struct types.
```

Therefore the documentation describes it as an:

```EXAMPLE / FLOW
anti-constraint
```

Important `allows ref struct` Rules

If:

```C#
where T :
    allows ref struct
```

then generic code must behave as though `T` **might** be a ref struct.

That means relevant ref-safety rules apply.

For example, such a value cannot simply be:

```EXAMPLE / FLOW
boxed
```

```
stored in an ordinary class field
*
allowed to escape into unsafe lifetime contexts
```


and the anti-constraint must appear last in that type parameter's constraint list.

Can `allows ref struct` Combine with `class`?

No.

This would be contradictory:

```C#
// where T : class, allows ref struct
```

because `ref struct` is not a reference type.

The language explicitly disallows combining `allows ref struct` with `class` or `class?`.

Do Normal Business Services Need `allows ref struct`?

Usually:

```EXAMPLE / FLOW
No.
```

It's most relevant to:

```EXAMPLE / FLOW
High-performance generic libraries

Span-based algorithms

Parsing

Buffer processing

Low-allocation APIs
```

For:

```EXAMPLE / FLOW
GiftService

IGiftRepository

CampaignService
```

ordinary generic constraints are normally enough.

Generic Constraints Enable Operations

Suppose:

```C#
public static int Compare<T>(
    T first,
    T second)
    where T :
        IComparable<T>
```

```
{
    return first.CompareTo(
        second);
}
```

Because of:

```
C#
where T :
    IComparable<T>
```

compiler knows:

```
C#
first.CompareTo(...)
```

is available.

Without Constraint

```
C#
public static int Compare<T>(
    T first,
    T second)
{
    // return first.CompareTo(
    //     second);
}
```

Compiler cannot assume:

```
EXAMPLE / FLOW
T implements CompareTo
```

Self-Referential Generic Constraint

This pattern:

```
C#
where T :
    IComparable<T>
```

is common.

Another advanced form:

```
C#
public interface IEntity<TSelf>
    where TSelf :
        IEntity<TSelf>
{
}
```

This is sometimes called a self-referential generic constraint.

It is useful in certain strongly typed APIs but can become overly complex if used without a clear benefit.

Static Abstract Generic Programming

From Topic 15, modern interfaces can define static abstract members.

Example conceptually:

```C#
public interface IFactory<TSelf>
    where TSelf :
        IFactory<TSelf>
{
    static abstract TSelf Create();
}
```

Then:

```C#
public static T Create<T>()
    where T :
        IFactory<T>
{
    return T.Create();
}
```

This is an important advanced use of generics in modern C# and forms part of .NET's generic-math architecture. Generic interfaces containing static abstract members are resolved using compile-time type information rather than ordinary object virtual dispatch.

Generic Repository Example

For learning:

```C#
public interface IEntity
{
    long Id
    {
        get;
    }
}
```

Generic contract:

```C#
public interface IRepository<TEntity>
    where TEntity :
        class,
        IEntity
{
    Task<TEntity?>
        FindAsync(
            long id,
            CancellationToken cancellationToken);

    Task AddAsync(
        TEntity entity,
        CancellationToken cancellationToken);
}
```

Implementation

```C#
public sealed class InMemoryRepository<TEntity> :
    IRepository<TEntity>
    where TEntity :
        class,
        IEntity
{
    private readonly Dictionary<long, TEntity>
        _items = [];

    public Task<TEntity?>
        FindAsync(
            long id,
            CancellationToken cancellationToken)
    {
        _items.TryGetValue(
            id,
            out TEntity? entity);

        return Task.FromResult(
            entity);
    }

    public Task AddAsync(
        TEntity entity,
        CancellationToken cancellationToken)
    {
        ArgumentNullException.ThrowIfNull(
            entity);

        _items[entity.Id] = entity;

        return Task.CompletedTask;
    }
}
```

Gift Repository

```C#
IRepository<Gift> giftRepository = new InMemoryRepository<Gift>();
```

Campaign:

```C#
IRepository<Campaign> campaignRepository = new InMemoryRepository<Campaign>();
```

Fund:

```C#
IRepository<Fund> fundRepository = new InMemoryRepository<Fund>();
```

Same generic infrastructure.

Different domain types.

Why Constraint Helps

Without:

```
C#
where TEntity :
    IEntity
```

this would not compile:

```
C#
entity.Id
```

With the constraint, the compiler knows every `TEntity` provides:

```
C#
Id
```

But Generic Repository Is Not Automatically Good Architecture

Generics make a generic repository easy to create.

That does **not** mean every EF Core application should automatically wrap every entity in:

```
C#
IRepository<TEntity>
```

Some entities may need:

```
EXAMPLE / FLOW
Different query behavior

Domain-specific operations

Aggregate-specific persistence

Different security rules

Different loading strategies
```

You will evaluate this architecture more carefully in:

```
EXAMPLE / FLOW
Topic 35
Repository, Unit of Work,
Service Layer and Mapping Patterns
```

For Topic 20, the important point is how the generic language mechanism works.

Generic Service Example

```
C#
public interface IValidator<T>
{
    void Validate(
```

```
        T value);
}
```


Gift validator:

```C#
public sealed class GiftValidator :
    IValidator<Gift>
{
    public void Validate(
        Gift gift)
    {
        ArgumentNullException.ThrowIfNull(
            gift);

        if (gift.TotalAmount <= 0m)
        {
            throw new InvalidOperationException(
                "Gift amount must be positive.");
        }
    }
}
```

Campaign validator:

```C#
public sealed class CampaignValidator :
    IValidator<Campaign>
{
    public void Validate(
        Campaign campaign)
    {
        ArgumentNullException.ThrowIfNull(
            campaign);
    }
}
```

One abstraction:

EXAMPLE / FLOW

```
IValidator<T>
```

many strongly typed implementations.

Open Generic Type

Consider:

```C#
typeof(
    Repository<>)
```

The type parameters aren't fully supplied.

This represents the generic type definition.

Conceptually:

EXAMPLE / FLOW

```
Repository<>
```

is open/unbound with respect to its generic argument.

Closed Generic Type

```C#
Repository<Gift>
```

is closed because all required type arguments have been supplied.

You can create instances:

```C#
Repository<Gift> repository = new();
```

Open vs Closed

```EXAMPLE / FLOW
Repository<>
    →
Generic definition
    →
Not enough type information
for ordinary instance use
```

```EXAMPLE / FLOW
Repository<Gift>
    →
Closed constructed type
    →
Can instantiate
```

The C# specification distinguishes generic definitions/unbound generic types from constructed types created by supplying type arguments.

Partially Open Generic Types

In advanced reflection/type-system scenarios, a generic constructed type can still contain type parameters.

Example conceptually:

```EXAMPLE / FLOW
Dictionary<string, T>
```

inside another generic context.

Not all type arguments are concrete yet.

Such types are still open with respect to unresolved type parameters.

Runtime Type Information Is Preserved

C#/.NET generics are not simply erased after compilation.

At runtime, reflection can distinguish types such as:

```
C#
List<int>

List<string>

Repository<Gift>
```

and inspect their actual generic arguments.

Microsoft explicitly notes that generic type information remains available at runtime.

typeof(T)

Inside generic code:

```
C#
public static void PrintType<T>()
{
    Console.WriteLine(
        typeof(T));
}
```

Call:

```
C#
PrintType<Gift>();
```

can determine runtime type metadata corresponding to:

```
EXAMPLE / FLOW
Gift
```

typeof(List<>)

You can inspect the unbound generic definition:

```
C#
Type type = typeof(List<>);
```

This means:

```
EXAMPLE / FLOW
The generic List type itself
```

not:

```
EXAMPLE / FLOW
List<int>
```

or:

```
EXAMPLE / FLOW
List<string>
```

C# 14 — nameof with Unbound Generic Types

Beginning with C# 14:

```
C#
nameof(
    List<>)
```

is valid and produces:

```
EXAMPLE / FLOW
"List"
```

Previously, `nameof` required a bound/closed generic form such as `List<int>`. This is one of C# 14's generic-related language improvements.

Generic Static Members

This is an important advanced behavior.

Consider:

```
C#
public static class Counter<T>
{
    public static int Value;
}
```

Now:

```
C#
Counter<Gift>.Value = 10;

Counter<Campaign>.Value = 20;
```

These are separate static values.

Why?

Each closed generic type has its own static state.

Conceptually:

```
EXAMPLE / FLOW
Counter<Gift>
    →
static Value = 10
```

```
EXAMPLE / FLOW
Counter<Campaign>
    →
static Value = 20
```

The C# type specification states that each closed constructed generic type has its own set of static variables.

Generic Static Cache Example

```
C#
public static class TypeMetadata<T>
{
```

```
    public static readonly string Name = typeof(T).Name;
}
```


Then:

```
C#
TypeMetadata<Gift>.Name
```

stores metadata for Gift.

```
C#
TypeMetadata<Campaign>.Name
```

belongs to another closed generic type.

This pattern can be useful for per-type cached metadata.

Be Careful with Static Generic Memory

If:

```
C#
public static class Cache<T>
{
    public static readonly Dictionary<long, T>
        Items = [];
}
```

then every used closed type gets its own cache:

```
EXAMPLE / FLOW
Cache<Gift>

Cache<Campaign>

Cache<Fund>
```

This can unintentionally create long-lived static references.

Generics do not eliminate static-state lifetime issues from Topic 8.

Generic Runtime Behavior

.NET runtime handling differs between value-type and reference-type generic instantiations.

For value types, the runtime creates specialized generic code for each unique value-type instantiation.

For reference-type instantiations, runtime code can generally be shared because references have compatible representation.

Conceptually:

```
EXAMPLE / FLOW
List<int>
    →
value-type specialization
```

EXAMPLE / FLOW

```
List<long>
    →
another value-type specialization
```

while reference-type uses can share more runtime implementation machinery.

Why Generics Can Improve Value-Type Performance

Compare an old object-based container:

C#

```
object value = 100;
```

The `int` must be boxed.

A generic:

C#

```
List<int>
```

can store `int` values in a type-aware representation without the ordinary object-boxing requirement.

Microsoft specifically identifies avoiding boxing for value-type generic collections as a performance advantage of generics.

Generic Does Not Mean "Always Faster"

Generics can improve:

EXAMPLE / FLOW

```
Type safety

Reuse

Value-type storage
```

but they can also introduce:

EXAMPLE / FLOW

```
More complex abstractions

Additional generated native code
for value-type instantiations

Harder-to-read constraint systems
```

Choose generics because the operation truly varies by type, not because generics look advanced.

Generic Algorithms

Example:

C#

```
public static void Swap<T>(
    ref T first,
    ref T second)
{
    T temp = first;
```

```
    first = second;

    second = temp;
}
```

Works for:

```C#
int

string

Gift

Campaign

GiftId
```

Usage

```C#
int first = 10;

int second = 20;

Swap(
    ref first,
    ref second);
```

or:

```C#
Gift firstGift =...;

Gift secondGift =...;

Swap(
    ref firstGift,
    ref secondGift);
```

One algorithm.

Many types.

Generic Equality Problem

Suppose:

```C#
public static bool Same<T>(
    T first,
    T second)
{
    // return first == second;
}
```

This generally doesn't compile for unconstrained `T`.

Why?

The compiler cannot assume:

EXAMPLE / FLOW

```
Every possible T
supports operator ==
```

Better Generic Equality

A common approach:

C#

```
public static bool Same<T>(
    T first,
    T second)
{
    return EqualityComparer<T>
        .Default
        .Equals(
            first,
            second);
}
```

This gives the runtime/framework an appropriate equality strategy for `T`.

EqualityComparer.Default

This is an important generic utility.

It uses type-specific equality behavior where available.

For example:

EXAMPLE / FLOW

```
int
    →
integer equality

string
    →
string equality

record
    →
record value equality

custom IEquatable<T>
    →
typed equality
```

Collections such as `Dictionary<TKey,TValue>` and `HashSet<T>` rely heavily on generic equality comparers.

You'll revisit this in Topic 21.

Generic Comparison

For ordering:

C#

```
public static int Compare<T>(
    T first,
```

```
    T second)
    where T :
        IComparable<T>
{
    return first.CompareTo(
        second);
}
```

Here the constraint is essential.

Generic Nullability

Nullability and generics interact in subtle ways.

Consider:

```C#
T?
```

The meaning depends partly on the constraints on `T`.

Examples:

```C#
where T :
    class
```

makes reference-type nullability meaningful.

```C#
where T :
    struct
```

has value-type/nullability rules.

Unconstrained nullable generic type parameters have additional language rules.

For normal business code, prefer explicit constraints when nullability semantics matter.

Example — Reference Entity

```C#
public interface IRepository<TEntity>
    where TEntity :
        class
{
    Task<TEntity?>
        FindAsync(
            long id);
}
```

This communicates:

```EXAMPLE / FLOW
TEntity is a reference type.

FindAsync may return null
when entity doesn't exist.
```

Example — Non-null Key

```C#
public sealed class Lookup<TKey, TValue>
    where TKey :
        notnull
{
}
```

This expresses:

EXAMPLE / FLOW

```
Null keys are not intended.
```

Constraints Are Part of Your API

Compare:

```C#
public class Box<T>
{
}
```

with:

```C#
public class Box<T>
    where T :
        class,
        IEntity,
        new()
{
}
```

The second is much more restrictive.

Only add constraints that your implementation requires.

Over-Constrained API

Bad:

```C#
public sealed class Validator<T>
    where T :
        class,
        IEntity,
        IAuditable,
        IComparable<T>,
        new()
{
}
```

if all validation actually needs is:

```C#
entity.Id
```

Then only:

```
C#
where T :
    IEntity
```

may be appropriate.

Under-Constrained API

Opposite mistake:

```
C#
public static void Save<T>(
    T entity)
{
    // Need TenantId,
    // but compiler knows nothing.
}
```

Developers then reach for:

```
C#
dynamic
```

or reflection.

Better:

```
C#
where T :
    ITenantScoped
```

if that capability is genuinely required.

Don't Replace Good Interfaces with Reflection

Bad generic code:

```
C#
public static long GetId<T>(
    T value)
{
    PropertyInfo? property = typeof(T).GetProperty("Id");

    return (long)
        property!
            .GetValue(
                value)!;
}
```

This loses:

```
EXAMPLE / FLOW
Compile-time safety

Refactoring safety

Performance
```

```
Clarity
```


Better:

```C#
public static long GetId<T>(
    T value)
    where T :
        IEntity
{
    return value.Id;
}
```

Generic Variance

Generic types are normally:

```EXAMPLE / FLOW
Invariant
```

unless their interface/delegate type parameters are declared variant.

Variance includes:

```EXAMPLE / FLOW
Covariance
    →
out

Contravariance
    →
in
```

Variance is supported for generic interfaces and delegates and applies to reference types, not value types.

Why Variance Matters

Suppose:

```C#
public class Entity
{
}

public class Gift :
    Entity
{
}
```

Clearly:

```EXAMPLE / FLOW
Gift
IS AN
Entity
```

But that doesn't automatically mean:

EXAMPLE / FLOW

```
List<Gift>
IS A
List<Entity>
```

List Is Invariant

This is invalid:

C#

```
List<Gift> gifts = [];

// List<Entity> entities =
//     gifts;
```

Why?

If this were allowed:

C#

```
entities.Add(
    new Campaign());
```

the original:

EXAMPLE / FLOW

```
List<Gift>
```

would suddenly contain a Campaign.

That would destroy type safety.

Generic Class Invariance

Even if:

EXAMPLE / FLOW

```
Gift → Entity
```

that doesn't imply:

EXAMPLE / FLOW

```
Generic<Gift>
    →
Generic<Entity>
```

unless the generic abstraction explicitly supports variance.

Covariance — out

Covariance allows:

EXAMPLE / FLOW

```
More derived generic source
    →
More general generic target
```

Example:

```C#
IEnumerable<Gift> gifts = [];

IEnumerable<Entity> entities = gifts;
```

This works because:

```EXAMPLE / FLOW
IEnumerable<out T>
```

is covariant.

Covariance Mental Model

```EXAMPLE / FLOW
Producer
    →
out T
```

An interface that only produces `T` can often safely be covariant.

Example:

```C#
public interface IEntitySource<out T>
{
    T Get();
}
```

Fundraising Covariance

```C#
public interface IEntitySource<out T>
{
    T Get();
}
```

Implementation:

```C#
public sealed class GiftSource :
    IEntitySource<Gift>
{
    public Gift Get()
    {
        return new Gift();
    }
}
```

Then:

```C#
IEntitySource<Gift>
    giftSource = new GiftSource();

IEntitySource<Entity>
    entitySource = giftSource;
```

Safe because consumer only receives entities from the source.

Why out

The generic type parameter is primarily flowing:

EXAMPLE / FLOW

```
OUT
of the interface
```

Example:

C#

```
T Get();
```

Hence:

C#

```
out T
```

Contravariance — in

Contravariance moves in the opposite direction.

Suppose:

C#

```
public interface IEntityHandler<in T>
{
    void Handle(
        T entity);
}
```

The type primarily consumes:

EXAMPLE / FLOW

```
T
```

So:

EXAMPLE / FLOW

```
in T
```

Example

C#

```
public sealed class EntityHandler :
    IEntityHandler<Entity>
{
    public void Handle(
        Entity entity)
    {
    }
}
```

Because it can handle **any** Entity, it can certainly handle a Gift.

Therefore:

```C#
IEntityHandler<Entity>
    entityHandler = new EntityHandler();

IEntityHandler<Gift>
    giftHandler = entityHandler;
```

can be valid due to contravariance.

Contravariance Mental Model

```EXAMPLE / FLOW
Consumer
    →
in T
```

A component capable of consuming a broad base type can also consume a derived type.

Built-In Examples

Covariant:

```EXAMPLE / FLOW
IEnumerable<out T>

IReadOnlyCollection<out T>

IReadOnlyList<out T>
```

Contravariant:

```EXAMPLE / FLOW
IComparer<in T>

IEqualityComparer<in T>

IComparable<in T>
```

Microsoft documents these as standard variant generic interfaces.

Variance Works Only for Reference Types

This works:

```C#
IEnumerable<string> strings = [];

IEnumerable<object> objects = strings;
```

because:

```EXAMPLE / FLOW
string
```

is a reference type.

But:

```C#
IEnumerable<int> numbers = [];

// IEnumerable<object> objects =
//     numbers;
```

does not work through variance because `int` is a value type.

Why Generic Classes Usually Don't Declare Variance

Variance modifiers:

```C#
in

out
```

are used on generic interfaces and delegates.

You don't write:

```C#
// class Box<out T>
```

C# doesn't support variance declarations on ordinary generic classes.

Generic Delegate Preview

Delegates can also be generic.

Examples you already encounter:

```C#
Action<Gift>

Func<Gift, bool>

Predicate<Gift>
```

These will be explored deeply in:

EXAMPLE / FLOW

```
Topic 22
Delegates, Action, Func and Predicate
```

But generics provide their type-safety mechanism.

Generic Async Return Values

```C#
Task<Gift>
```

is a generic type.

Here:

EXAMPLE / FLOW

```
T = Gift
```

The task eventually produces:

EXAMPLE / FLOW

```
Gift
```

Another:

C#

```
Task<IReadOnlyList<Gift>>
```

where `T` is itself:

EXAMPLE / FLOW

```
IReadOnlyList<Gift>
```

Generics can nest freely.

Nested Generics

Example:

C#

```
Dictionary<
    long,
    List<Gift>>
```

Read:

EXAMPLE / FLOW

```
Dictionary
    ↓
Key = long
Value = List<Gift>
```

Another:

C#

```
Task<
    IReadOnlyList<
        GiftResponse>>
```

Read inside-out:

EXAMPLE / FLOW

```
GiftResponse

IReadOnlyList<GiftResponse>

Task<IReadOnlyList<GiftResponse>>
```

Generic Type Aliases?

C# doesn't create a new runtime type merely through:

C#

```
using GiftMap = Dictionary<long, Gift>;
```

This creates an alias for source-code convenience.

The underlying type remains:

EXAMPLE / FLOW

```
Dictionary<long, Gift>
```

Generics vs object

object

```C#
public object Get(
    object value)
{
    return value;
}
```

Usage requires casts:

```C#
Gift gift = (Gift)Get(originalGift);
```

Generic

```C#
public T Get<T>(
    T value)
{
    return value;
}
```

Usage:

```C#
Gift gift = Get(originalGift);
```

No cast.

Compiler preserves type information.

Generics vs dynamic

`dynamic` moves type resolution to runtime.

```C#
dynamic value = gift;

value.SomeMethodThatMayNotExist();
```

may compile and fail later.

Generic:

```C#
public void Process<T>(
    T value)
    where T :
        IGiftProcessable
```

provides compile-time constraints.

For most reusable strongly typed APIs, generics are preferable to `dynamic`.

Generics vs Method Overloading

Without generics:

```C#
void Print(
    Gift gift)

void Print(
    Campaign campaign)

void Print(
    Fund fund)
```

If behavior is truly identical:

```C#
void Print<T>(
    T value)
```

may reduce duplication.

But if each type requires genuinely different behavior, polymorphism or overloads may be clearer.

Don't Use Generic Merely to Remove Three Lines

Bad:

```C#
public void Process<T>(
    T value)
{
    if (
        value is Gift gift)
    {
        ProcessGift(
            gift);
    }
    else if (
        value is Campaign campaign)
    {
        ProcessCampaign(
            campaign);
    }
}
```

This generic method isn't gaining much.

It still contains type-specific branching.

Better Alternatives

Maybe use:

```EXAMPLE / FLOW
Overloads
```

```
Interface polymorphism
*
Strategy

Separate services
```


depending on the domain.

Generics should model **type-parameterized behavior**, not hide unrelated code behind `<T>`.

Generic Factory Example

C#

```
public static class Factory
{
    public static T Create<T>()
        where T :
            new()
    {
        return new T();
    }
}
```

Usage:

C#

```
GiftDraft draft = Factory.Create< GiftDraft>();
```

Generic Constraint + Interface Factory

A more expressive modern design can avoid requiring a parameterless constructor:

C#

```
public interface IFactory<TSelf>
    where TSelf :
        IFactory<TSelf>
{
    static abstract TSelf Create();
}
```

This permits each type to decide how creation works.

It is more advanced and should only be used where generic static behavior genuinely matters.

Fundraising Generic Result Example

C#

```
public sealed record Result<T>(
    bool IsSuccess,
    T? Value,
    string? Error)
{
    public static Result<T>
        Success(
            T value)
    {
        return new(
            true,
            value,
            null);
```

```
    }

    public static Result<T>
        Failure(
            string error)
    {
        return new(
            false,
            default,
            error);
    }
}
```

Usage:

C#
```
Result<Gift> result = Result<Gift>.Success(gift);
```

Another:

C#
```
Result<Campaign>
```

Another:

C#
```
Result<long>
```

Caution with Result and T?

Nullable semantics can become subtle for:

EXAMPLE / FLOW
```
T = value type

T = reference type

T = nullable type
```

Production result types often explicitly model:

EXAMPLE / FLOW
```
Success value

Failure state
```

rather than relying only on `default(T)` to indicate absence.

Generic Tenant-Safe Example

C#
```
public interface ITenantScoped
{
    long TenantId
    {
        get;
    }
}
```

Generic validator:

```C#
public static class TenantGuard
{
    public static void EnsureTenant<T>(
        T value,
        long expectedTenantId)
        where T :
            ITenantScoped
    {
        ArgumentNullException
            .ThrowIfNull(
                value);

        if (
            value.TenantId
            !=
            expectedTenantId)
        {
            throw new InvalidOperationException(
                "Cross-tenant access is not allowed.");
        }
    }
}
```

Usage:

```C#
TenantGuard.EnsureTenant(
    gift,
    currentTenantId);
```

Also:

```C#
TenantGuard.EnsureTenant(
    campaign,
    currentTenantId);
```

Security Lesson

Generics provide:

```EXAMPLE / FLOW
Type safety
```

They do **not** provide:

```EXAMPLE / FLOW
Authorization

Tenant filtering

Input validation

Authentication
```

A generic repository:

```
C#
IRepository<TEntity>
```

does not automatically prevent:

EXAMPLE / FLOW

```
Cross-tenant access
```

You must enforce security rules explicitly.

Bad Generic Repository Security

```
C#
public Task<TEntity?>
    FindAsync(
        long id)
{
    return _dbSet.FindAsync(
        id);
}
```

If entities are tenant-scoped, this might retrieve an entity belonging to another tenant unless the architecture guarantees filtering elsewhere.

Generic abstraction must not erase important security semantics.

Better Domain-Specific Contract

Sometimes:

```
C#
IGiftRepository
```

is better than:

```
C#
IRepository<Gift>
```

because it can expose:

```
C#
Task<Gift?>
    FindForTenantAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken);
```

The API itself communicates tenant-aware access.

This is why generic reuse must not override domain correctness.

Generic Constraints and Security

This:

```C#
where T :
    ITenantScoped
```

ensures:

EXAMPLE / FLOW

```
T has TenantId
```

It does not ensure:

EXAMPLE / FLOW

```
TenantId is authorized
```

You still need:

```C#
if (
    entity.TenantId
    !=
    currentTenantId)
{
    ...
}
```

Generic constraints establish capabilities—not trust.

Performance — Type Safety

One of the strongest practical benefits is eliminating:

EXAMPLE / FLOW

```
Repeated runtime casts
```

and catching many mistakes during compilation.

That improves both reliability and maintainability.

Performance — Boxing

Generic collections are particularly beneficial for value types.

Example:

```C#
List<int>
```

does not need the same normal boxing behavior that an object-based collection would require for every integer.

Performance — Runtime Specialization

For value-type generic instantiations, the runtime can create specialized code.

For reference-type instantiations, runtime code is generally shared more extensively.

This is one reason generic performance behavior can differ based on the supplied type argument.

Performance — Code Size Tradeoff

If your generic algorithm is instantiated with many value types:

EXAMPLE / FLOW

```
Generic<int>

Generic<long>

Generic<decimal>

Generic<MyStruct1>

Generic<MyStruct2>
```

the runtime may generate specialized native code for these value-type instantiations.

That can increase code size.

This is usually a good tradeoff for type safety/performance, but matters in highly optimized libraries.

Performance — Constraints May Enable Better Code

A constraint can avoid:

EXAMPLE / FLOW

```
Reflection

dynamic dispatch

object casting
```

Example:

C#

```
where T :
    IComparable<T>
```

allows direct typed calls to:

C#

```
CompareTo
```

Performance — Generic Abstraction Is Usually Cheap

Don't avoid:

C#

```
List<T>

Dictionary<TKey,TValue>

Task<T>

IEnumerable<T>
```

because:

EXAMPLE / FLOW

Generics sound expensive.

TOPIC 20 · C#

Generics are deeply integrated into the CLR and C# type system.

Common Mistake — Using object Instead of Generics

Bad:

```C#
public object GetValue(
    object value)
```

when:

```C#
public T GetValue<T>(
    T value)
```

would preserve type information.

Common Mistake — Using dynamic Instead of Constraints

Bad:

```C#
public void Process<T>(
    T value)
{
    dynamic item = value!;

    Console.WriteLine(
        item.TenantId);
}
```

Better:

```C#
public void Process<T>(
    T value)
    where T :
        ITenantScoped
{
    Console.WriteLine(
        value.TenantId);
}
```

Common Mistake — Assuming T Has Members

This doesn't compile:

```C#
public void Print<T>(
    T value)
{
    // Console.WriteLine(
    //     value.Name);
}
```

unless constraints guarantee `Name`.

Common Mistake — Over-Constraining

Don't add:

```C#
where T :
    class,
    new()
```

if the implementation doesn't require either capability.

Constraints become part of the public contract.

Common Mistake — Under-Constraining

If generic code needs:

```C#
TenantId
```

don't use reflection.

Require:

```C#
where T :
    ITenantScoped
```

Common Mistake — Assuming `struct` Includes Nullable

It doesn't.

```C#
where T :
    struct
```

requires a non-nullable value type.

Common Mistake — Combining struct and new()

Invalid:

```C#
// where T : struct, new()
```

The language prohibits it.

Common Mistake — `new()` Means Any Constructor

No.

It requires an accessible public parameterless constructor.

Common Mistake — Using new() Only Because Factory Needs Creation

Sometimes:

```
C#
Func<T>
```

or a factory interface is more flexible than forcing every type to expose a public parameterless constructor.

Example:

```
C#
public sealed class Factory<T>
{
    private readonly Func<T>
        _create;

    public Factory(
        Func<T> create)
    {
        _create = create;
    }

    public T Create()
    {
        return _create();
    }
}
```

This can support parameterized/internal creation logic.

Common Mistake — `default(T)` Means Missing

Not necessarily.

For:

```
EXAMPLE / FLOW
int
```

default is:

```
EXAMPLE / FLOW
0
```

which may be valid.

For:

```
EXAMPLE / FLOW
bool
```

default is:

```
EXAMPLE / FLOW
false
```

which may be valid.

Model absence explicitly when necessary.

Common Mistake — List Converts to List

It does not.

`List<T>` is invariant.

Use a covariant interface such as:

```
C#
IEnumerable<Entity>
```

where the operation permits it.

Common Mistake — Variance with Value Types

Variance applies to reference types.

This doesn't work:

```
C#
IEnumerable<int>
    →
IEnumerable<object>
```

through generic covariance.

Common Mistake — out Means out Parameter

In:

```
C#
interface IProducer<out T>
```

`out` means:

```
EXAMPLE / FLOW
Covariant generic type parameter
```

This is conceptually different from:

```
C#
void TryGet(
    out T value)
```

method parameter syntax.

Common Mistake — in Means Method in Parameter

Similarly:

```
C#
interface IConsumer<in T>
```

means:

It is distinct from:

```
C#
void Process(
    in T value)
```

readonly-by-reference method parameters.

Common Mistake — Generic Means Reusable Automatically

Example:

```
C#
public class Processor<T>
{
    public void Process(
        T value)
    {
        if (value is Gift)
        {
        }

        if (value is Fund)
        {
        }

        if (value is Campaign)
        {
        }
    }
}
```

This is generic syntax but poor generic abstraction.

Common Mistake — Generic Repository for Every Entity

A single:

```
C#
Repository<TEntity>
```

may look elegant but can hide important domain differences.

Examples:

```
EXAMPLE / FLOW
Gift requires allocations.

Campaign requires date-range queries.

Constituent requires address/contact queries.

Tenant isolation differs.

Aggregate boundaries differ.
```

Use generics where behavior is genuinely common.

Common Mistake — Forgetting Static Fields Are Per Closed Type

```C#
Cache<Gift>.Items
```

is separate from:

```C#
Cache<Campaign>.Items
```

This can surprise developers who expected one global static cache.

Common Mistake — Forgetting Runtime Generic Type Information

Unlike type-erased generic systems, .NET can inspect:

```C#
typeof(
    List<Gift>)
```

and retrieve actual generic type arguments through reflection.

Common Mistake — Using allows ref struct Without Need

Don't write:

```C#
where T :
    allows ref struct
```

in ordinary repository/service code.

It forces the generic implementation to obey stricter ref-safety rules.

Use it only when supporting ref-struct arguments is genuinely part of the API.

Practical Exercise 1 — Generic Box

Create:

```C#
public sealed class Box<T>
{
    public Box(
        T value)
    {
        Value = value;
    }

    public T Value
    {
        get;
    }
}
```

Use:

```C#
Box<Gift> giftBox = new(gift);

Box<int> numberBox = new(100);
```

Practical Exercise 2 — Generic Method

```C#
public static T Identity<T>(
    T value)
{
    return value;
}
```

Call:

```C#
Gift gift2 = Identity(gift);
```

Compiler infers:

EXAMPLE / FLOW

```
T = Gift
```

Practical Exercise 3 — Interface Constraint

```C#
public interface IHasId
{
    long Id
    {
        get;
    }
}
```

Write:

```C#
public static long GetId<T>(
    T value)
    where T :
        IHasId
{
    return value.Id;
}
```

Practical Exercise 4 — new()

```C#
public static T Create<T>()
    where T :
        new()
{
    return new T();
}
```

Ask:

Would this work?

```C#
public sealed class Gift
{
    public Gift(
        decimal amount)
    {
    }
}
```

Answer:

EXAMPLE / FLOW

```
No
```

because no public parameterless constructor exists.

Practical Exercise 5 — default

```C#
static T GetDefault<T>()
{
    return default;
}
```

Results:

EXAMPLE / FLOW

```
GetDefault<int>()
    →
0

GetDefault<bool>()
    →
false

GetDefault<Gift>()
    →
null
```

Practical Exercise 6 — Generic Repository

```C#
public interface IEntity
{
    long Id
    {
        get;
    }
}
```

Create:

```C#
Repository<Gift>

Repository<Campaign>
```

from:

```C#
Repository<TEntity>
    where TEntity :
        IEntity
```

Practical Exercise 7 — Covariance

Given:

```EXAMPLE / FLOW
Gift
IS AN
Entity
```

Which is valid?

```C#
IEnumerable<Gift> gifts = [];

IEnumerable<Entity> entities = gifts;
```

Yes.

Because:

```EXAMPLE / FLOW
IEnumerable<out T>
```

is covariant.

Practical Exercise 8 — Invariance

Is this valid?

```C#
List<Gift> gifts = [];

// List<Entity> entities =
//     gifts;
```

No.

`List<T>` is invariant.

Practical Exercise 9 — Contravariance

```C#
public interface IHandler<in T>
{
    void Handle(
```

```
        T value);
}
```


An:

```
EXAMPLE / FLOW
IHandler<Entity>
```

can be used where:

```
EXAMPLE / FLOW
IHandler<Gift>
```

is needed because a handler capable of handling every Entity can handle a Gift.

Practical Exercise 10 — Strongly Typed Generic Result

```
C#
Result<Gift>

Result<Campaign>

Result<IReadOnlyList<Fund>>
```

all reuse one generic result definition.

Practical Exercise 11 — Tenant Constraint

```
C#
public static bool IsTenant<T>(
    T entity,
    long tenantId)
    where T :
        ITenantScoped
{
    return entity.TenantId
        == tenantId;
}
```

Use for:

```
EXAMPLE / FLOW
Gift

Campaign

Fund

Constituent
```

without losing compile-time safety.

Practical Exercise 12 — Static Generic State

```
C#
public static class Count<T>
{
```

```
    public static int Value;
}
```


Then:

```C#
Count<Gift>.Value = 10;

Count<Campaign>.Value = 20;
```

What is:

```C#
Count<Gift>.Value
```

?

EXAMPLE / FLOW

```
10
```

Each closed generic type owns separate static state.

Interview Questions

1. What are generics in C#?

Generics allow types and methods to define placeholders for one or more types.

The caller supplies concrete type arguments later.

Example:

```C#
List<Gift>
```

uses the generic definition:

```C#
List<T>
```

with:

EXAMPLE / FLOW

```
T = Gift
```

Generics improve code reuse and compile-time type safety.

2. What is a type parameter?

A placeholder declared by a generic type or method.

Example:

```C#
Repository<TEntity>
```

`TEntity` is the type parameter.

3. What is a type argument?

The actual type supplied for a type parameter.

```C#
Repository<Gift>
```

`Gift` is the type argument.

4. What is a constructed generic type?

A generic type after actual type arguments have been supplied.

Example:

```C#
Dictionary<long, Gift>
```

is a constructed generic type.

5. What is the difference between `Repository<T>` and `Repository<Gift>`?

EXAMPLE / FLOW

```
Repository<T>
    →
Generic definition

Repository<Gift>
    →
Closed constructed type
```

6. What are the main benefits of generics?

Major benefits include:

EXAMPLE / FLOW

```
Compile-time type safety

Code reuse

Fewer explicit casts

Reduced InvalidCastException risk

Avoiding boxing in many value-type scenarios
```

7. Why are generics better than using object everywhere?

`object` loses the original specific type from the API surface and often requires casting.

Generics preserve the exact type through compilation and runtime metadata.

8. What is a generic method?

A method declaring its own type parameter list.

Example:

```
C#
T Echo<T>(
    T value)
```

9. Is a method automatically generic because it is inside a generic class?

No.

```
C#
class Repository<T>
{
    T Get()
}
```

`Get` uses the class's type parameter but declares no type parameters of its own.

A method becomes generic when it declares its own generic type parameters.

10. What is generic type inference?

The compiler determines the method's type argument from arguments at the call site.

Example:

```
C#
Echo(
    10);
```

infers:

```
EXAMPLE / FLOW
T = int
```

11. What are generic constraints?

Rules limiting which type arguments can be used for a generic parameter.

Example:

```
C#
where T :
    IEntity
```

means every valid `T` must implement or derive from the required contract.

12. Why are constraints useful?

They serve two purposes:

```
EXAMPLE / FLOW
Restrict invalid type arguments

Allow generic code to use
members guaranteed by the constraint
```

13. What does `where T : class` mean?

`T` must be a reference type.

With nullable-reference analysis, it represents a non-nullable reference-type constraint.

14. What does `where T : class?` mean?

`T` must be a reference type, and nullable reference type arguments are permitted under nullable-reference rules.

15. What does `where T : struct` mean?

`T` must be a non-nullable value type.

It doesn't include `Nullable<T>`.

16. What does `where T : notnull` mean?

`T` is intended to be a non-nullable value or reference type.

It is often useful for keys and other values that shouldn't represent null.

17. What does `where T : unmanaged` mean?

`T` must be a non-nullable unmanaged value type whose representation doesn't contain managed references.

It is useful mainly for low-level memory/interoperability scenarios.

18. What does a base-class constraint do?

Example:

```
C#
where T :
    Entity
```

requires every `T` to derive from `Entity`, allowing the generic implementation to use members declared by `Entity`.

19. What does an interface constraint do?

Example:

```
C#
where T :
    ITenantScoped
```

requires every `T` to implement that capability.

20. What does `new()` mean?

`T` must provide an accessible public parameterless constructor.

Then generic code can write:

```
C#
new T();
```

21. Where must `new()` appear in a constraint list?

It must come after the ordinary type/base/interface constraints.

Anti-constraints such as `allows ref struct` appear after it where applicable.

22. Can `new()` be combined with `struct`?

No.

The language prohibits combining them because the value-type constraint already supplies the relevant construction semantics.

23. What is `default(T)`?

It produces the default value for `T`.

Examples:

EXAMPLE / FLOW

```
T = int
    →
0

T = bool
    →
false

T = Gift
    →
null
```

24. What is the difference between `default(T)` and `where T : default`?

`default(T)` is an expression returning T's default value.

`where T : default` is a specialized generic constraint mainly for overrides and explicit interface implementations involving unconstrained nullable type parameters.

25. What is `allows ref struct`?

A C# 13 anti-constraint that permits a type parameter to receive a `ref struct` type argument.

Generic code must then obey ref-safety rules because `T` might be a ref struct.

26. Why can't `allows ref struct` be combined with `class`?

Because `class` requires a reference type, while a `ref struct` is a stack-restricted value type.

The constraints would conflict.

27. What is an open generic type?

A generic type that still contains unresolved type parameters.

Example:

C#

```
typeof(
    Repository<>)
```

represents the generic definition.

28. What is a closed generic type?

A generic type with all required type arguments supplied.

Example:

```C#
Repository<Gift>
```

It can be instantiated normally.

29. Are generic type arguments erased at runtime in .NET?

No.

.NET retains generic type information so reflection can inspect actual generic types and type arguments at runtime.

30. What happens to generics with value types at runtime?

The runtime creates specialized generic code for unique value-type instantiations.

For example:

EXAMPLE / FLOW

```
Generic<int>

Generic<long>
```

can receive different specialized runtime implementations.

31. What happens with reference-type generic instantiations?

Runtime implementation code can generally be shared across reference-type arguments because reference representations are compatible.

32. Why can generics reduce boxing?

A generic type such as:

```C#
List<int>
```

can work directly with `int` values instead of storing each value as `object`.

That avoids ordinary boxing/unboxing overhead.

33. Are static fields shared across all generic type arguments?

No.

Each closed generic type has its own static state.

```C#
Cache<Gift>
```

and:

```C#
Cache<Campaign>
```

have separate static fields.

34. What is generic invariance?

It means that even if:

EXAMPLE / FLOW

```
Gift
IS AN
Entity
```

TOPIC 20 · C#

you cannot automatically convert:

EXAMPLE / FLOW

```
Generic<Gift>
```

to:

EXAMPLE / FLOW

```
Generic<Entity>
```

Example:

C#

```
List<Gift>
```

cannot be assigned to:

C#

```
List<Entity>
```

35. What is covariance?

Covariance permits using a more-derived generic type argument where a less-derived one is expected.

Example:

C#

```
IEnumerable<Gift>
```

can be used as:

C#

```
IEnumerable<Entity>
```

because `IEnumerable<out T>` is covariant.

36. What does `out T` mean in a generic interface?

It declares a covariant type parameter.

The type is generally used in output/return positions.

Think:

EXAMPLE / FLOW

```
Producer
    →
out
```

37. What is contravariance?

Contravariance permits using a less-derived generic type argument where a more-derived one is required for consumer-like interfaces.

Example:

EXAMPLE / FLOW

```
IHandler<Entity>
```

may satisfy:

EXAMPLE / FLOW

```
IHandler<Gift>
```

if the interface declares:

C#

```
IHandler<in T>
```

38. What does `in T` mean in a generic interface?

It declares a contravariant type parameter.

Think:

EXAMPLE / FLOW

```
Consumer
    →
in
```

39. Does generic variance work with value types?

No.

Variance conversions apply to reference types.

40. Why is `List<T>` invariant?

Because it both:

EXAMPLE / FLOW

```
Produces T

and

Consumes T
```

Allowing `List<Gift>` to become `List<Entity>` would let callers add a non-Gift entity into a Gift list.

Best Practices

Use generics when:

EXAMPLE / FLOW

```
Same algorithm or abstraction
genuinely applies to many types.
```

Prefer:

C#

```
List<Gift>
```

over object-based collections.

Use descriptive type parameter names such as:

EXAMPLE / FLOW

```
TEntity

TKey

TValue

TResult
```

when they improve readability.

Use generic constraints to describe actual requirements.

Prefer:

```C#
where T :
    ITenantScoped
```

over reflection or `dynamic` when the capability is known.

Do not add constraints that your generic implementation doesn't need.

Remember:

```C#
where T :
    class
```

and:

```C#
where T :
    struct
```

have very different nullability/value semantics.

Use:

```C#
notnull
```

when null type arguments are inappropriate.

Use:

```C#
unmanaged
```

only for genuine low-level memory/interoperability requirements.

Use:

```C#
new()
```

only if generic code genuinely creates `T`.

Remember `new()` means:

not any constructor.

Use:

```C#
default
```

carefully.

Do not assume default means:

```
EXAMPLE / FLOW
Missing
```

for all types.

Prefer:

```C#
EqualityComparer<T>.Default
```

for generic equality rather than assuming `==` exists.

Use:

```C#
IComparable<T>
```

or another appropriate contract when generic ordering is required.

Understand:

```
EXAMPLE / FLOW
List<T>
```

is invariant.

Use covariant read-only/producer interfaces such as:

```C#
IEnumerable<T>
```

where appropriate.

Think:

```
EXAMPLE / FLOW
out = producer

in = consumer
```

for variance.

Remember generic variance only applies to reference types.

Remember each closed generic type has its own static fields.

Use generic repositories carefully.

Do not let a generic abstraction erase:

EXAMPLE / FLOW

```
Tenant security

Aggregate boundaries

Domain-specific behavior

Authorization requirements
```

Don't use reflection simply because the code is generic.

Don't use `dynamic` to avoid designing correct generic constraints.

Use:

```C#
allows ref struct
```

only for advanced ref-safe generic algorithms that genuinely need to support ref structs.

Take advantage of C# 14 features such as:

```C#
nameof(
    List<>)
```

when useful in generic-library/metaprogramming code.

Quick Revision

Concept	Meaning
Generic	Type-parameterized code
`T`	Type parameter
Type argument	Concrete supplied type
Generic definition	`Repository<T>`
Constructed type	`Repository<Gift>`
Generic method	Method with own `<T>`
Type inference	Compiler determines method type argument
Constraint	Restriction/capability required of `T`
`class`	Reference-type constraint
`class?`	Nullable/non-null reference-type constraint
`struct`	Non-nullable value-type constraint
`notnull`	Non-null type argument
`unmanaged`	Unmanaged value-type constraint
Base constraint	`T` derives from base type
Interface constraint	`T` implements interface
`new()`	Public parameterless constructor

Concept	Meaning
`default(T)`	Default value for `T`
`default` constraint	Specialized override/interface constraint
`allows ref struct`	Allows ref-struct type arguments
Open generic	Still contains unresolved type parameters
Closed generic	All type arguments supplied
Invariance	No generic conversion by inheritance alone
Covariance	`out T`, producer
Contravariance	`in T`, consumer
Generic static state	Separate per closed generic type

Complete Fundraising Example

Consider a family of tenant-scoped entities.

C#
```
public interface IEntity
{
    long Id
    {
        get;
    }
}
```

C#
```
public interface ITenantScoped
{
    long TenantId
    {
        get;
    }
}
```

Gift

C#
```
public sealed class Gift :
    IEntity,
    ITenantScoped
{
    public Gift(
        long id,
        long tenantId,
        decimal amount)
    {
        if (id <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(id));
        }

        if (tenantId <= 0)
```

```
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        Id = id;

        TenantId = tenantId;

        Amount = amount;
    }

    public long Id
    {
        get;
    }

    public long TenantId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }
}
```

Campaign

C#

```
public sealed class Campaign :
    IEntity,
    ITenantScoped
{
    public Campaign(
        long id,
        long tenantId,
        string name)
    {
        Id = id;

        TenantId = tenantId;

        Name = name;
    }

    public long Id
    {
        get;
    }

    public long TenantId
    {
        get;
    }
```

```
    public string Name
    {
        get;
    }
}
```

Generic Repository Contract

C#

```
public interface IRepository<TEntity>
    where TEntity :
        class,
        IEntity,
        ITenantScoped
{
    Task<TEntity?>
        FindAsync(
            long tenantId,
            long id,
            CancellationToken cancellationToken);

    Task AddAsync(
        TEntity entity,
        CancellationToken cancellationToken);

    Task<IReadOnlyList<TEntity>>
        GetAllAsync(
            long tenantId,
            CancellationToken cancellationToken);
}
```

Notice the constraints:

EXAMPLE / FLOW

```
class
    →
TEntity must be a reference type
```

EXAMPLE / FLOW

```
IEntity
    →
TEntity has Id
```

EXAMPLE / FLOW

```
ITenantScoped
    →
TEntity has TenantId
```

In-Memory Implementation

C#

```
public sealed class InMemoryRepository<TEntity> :
    IRepository<TEntity>
    where TEntity :
        class,
        IEntity,
        ITenantScoped
{
```

```
    private readonly Dictionary<long, TEntity>
        _entities = [];

    public Task<TEntity?>
        FindAsync(
            long tenantId,
            long id,
            CancellationToken cancellationToken)
    {
        cancellationToken
            .ThrowIfCancellationRequested();

        if (
            _entities.TryGetValue(
                id,
                out TEntity? entity)
            &&
            entity.TenantId
            == tenantId) { return Task.FromResult<TEntity?>(entity);
        }

        return Task.FromResult<TEntity?>(
            null);
    }

    public Task AddAsync(
        TEntity entity,
        CancellationToken cancellationToken)
    {
        ArgumentNullException.ThrowIfNull(
            entity);

        cancellationToken
            .ThrowIfCancellationRequested();

        _entities[entity.Id] = entity;

        return Task.CompletedTask;
    }

    public Task<IReadOnlyList<TEntity>>
        GetAllAsync(
            long tenantId,
            CancellationToken cancellationToken)
    {
        cancellationToken
            .ThrowIfCancellationRequested();

        List<TEntity> results = [];

        foreach (
            TEntity entity
            in _entities.Values)
        {
            if (
                entity.TenantId
                == tenantId) { results.Add(entity);
            }
        }

        return Task.FromResult<
            IReadOnlyList<TEntity>>(
                results);
    }
}
```

Gift Repository Usage

```C#
IRepository<Gift> gifts = new InMemoryRepository<Gift>();
```

Campaign:

```C#
IRepository<Campaign> campaigns = new InMemoryRepository<Campaign>();
```

The generic implementation is reused.

Type Safety

This compiles:

```C#
await gifts.AddAsync(
    gift,
    cancellationToken);
```

This doesn't:

```C#
// await gifts.AddAsync(
//     campaign,
//     cancellationToken);
```

because:

```
EXAMPLE / FLOW
IRepository<Gift>
```

accepts:

```
EXAMPLE / FLOW
Gift
```

not Campaign.

Generic Tenant Guard

```C#
public static class TenantGuard
{
    public static void EnsureTenant<T>(
        T entity,
        long expectedTenantId)
        where T :
            ITenantScoped
    {
        ArgumentNullException
            .ThrowIfNull(
                entity);

        if (
            entity.TenantId
            !=
```

```
            expectedTenantId)
        {
            throw new InvalidOperationException(
                "Entity does not belong to the current tenant.");
        }
    }
}
```

Gift:

```C#
TenantGuard.EnsureTenant(
    gift,
    currentTenantId);
```

Campaign:

```C#
TenantGuard.EnsureTenant(
    campaign,
    currentTenantId);
```

Same algorithm.

Different types.

Guaranteed contract:

EXAMPLE / FLOW

```
ITenantScoped
```

Generic Response

```C#
public sealed record ApiResult<T>(
    bool Success,
    T? Data,
    string? Error);
```

Gift:

```C#
ApiResult<GiftResponse>
```

List:

```C#
ApiResult<
    IReadOnlyList<
        GiftResponse>>
```

Generic Equality Helper

```C#
public static class GenericEquality
{
    public static bool AreEqual<T>(
        T first,
```

```
        T second)
    {
        return EqualityComparer<T>
            .Default
            .Equals(
                first,
                second);
    }
}
```

Use:

```C#
bool sameGiftId = GenericEquality.AreEqual(giftId1, giftId2);
```

Generic Comparison Helper

```C#
public static class GenericComparison
{
    public static T Max<T>(
        T first,
        T second)
        where T :
            IComparable<T>
    {
        return first.CompareTo(
            second)
            >= 0 ? first : second;
    }
}
```

Use:

```C#
decimal maxAmount = GenericComparison.Max(100m, 250m);
```

Complete Generics Decision Flow

```EXAMPLE / FLOW
Need same behavior
for more than one type?
        ↓
No
    →
Normal non-generic code
```

If yes:

```EXAMPLE / FLOW
Can behavior be expressed
without knowing concrete type?
        ↓
Yes
    →
Generic type/method
```

Then ask:

EXAMPLE / FLOW

```
Does generic code require
specific capabilities?
```

No:

EXAMPLE / FLOW

```
Unconstrained T
```

Yes:

EXAMPLE / FLOW

```
Add minimum necessary constraint
```

Constraint Decision Flow

EXAMPLE / FLOW

```
Need reference type?
    →
where T : class
```

EXAMPLE / FLOW

```
Need nullable-reference allowance?
    →
where T : class?
```

EXAMPLE / FLOW

```
Need non-nullable value type?
    →
where T : struct
```

EXAMPLE / FLOW

```
Need any non-null type?
    →
where T : notnull
```

EXAMPLE / FLOW

```
Need low-level unmanaged value?
    →
where T : unmanaged
```

EXAMPLE / FLOW

```
Need base-class members?
    →
where T : Base
```

EXAMPLE / FLOW

```
Need capability?
    →
where T : IInterface
```

EXAMPLE / FLOW

```
Need new T()?
    →
where T : new()
```

EXAMPLE / FLOW

```
Need Span/ref struct support?
    →
where T : allows ref struct
```

Variance Decision Flow

Ask:

EXAMPLE / FLOW

```
Does interface only produce T?
```

Yes:

EXAMPLE / FLOW

```
out T
    →
Covariance
```

Example:

C#

```
IEnumerable<out T>
```

Ask:

EXAMPLE / FLOW

```
Does interface only consume T?
```

Yes:

EXAMPLE / FLOW

```
in T
    →
Contravariance
```

Example:

C#

```
IComparer<in T>
```

If it both reads and writes `T`:

EXAMPLE / FLOW

```
Usually invariant
```

Example:

C#

```
List<T>
```

Generic Runtime Flow

EXAMPLE / FLOW

```
Generic Definition
Repository<T>
    ↓
Supply type argument
    ↓
Repository<Gift>
    ↓
Closed constructed type
    ↓
Runtime retains generic type info
```

For value type:

EXAMPLE / FLOW

```
Generic<int>
    ↓
Specialized runtime code
```

For reference types:

EXAMPLE / FLOW

```
Generic<Gift>
Generic<Campaign>
    ↓
Runtime can share more implementation code
```

Final Mental Model

Remember:

EXAMPLE / FLOW

```
T
    →
Unknown type placeholder
```

Constraint:

EXAMPLE / FLOW

```
where T : ...
    →
What the compiler is allowed
to assume about T
```

Type argument:

EXAMPLE / FLOW

```
Gift
    →
Actual type supplied
```

Generic definition:

EXAMPLE / FLOW

```
Repository<T>
```

Closed type:

EXAMPLE / FLOW

```
Repository<Gift>
```

Variance:

EXAMPLE / FLOW

```
out
    →
Producer

in
    →
Consumer
```

And the most important principle:

> **Generics let you reuse algorithms and abstractions without giving up compile-time type safety. Constraints should express only the capabilities the implementation genuinely requires. Generics are most effective when they model behavior that is truly independent of a specific concrete type—not when they are used merely to hide unrelated type-specific logic.**

For the Fundraising Platform, good generic candidates include concepts such as:

EXAMPLE / FLOW

```
Result<T>

IValidator<T>

IReadOnlyList<T>

Task<T>

typed comparison/equality helpers

carefully designed repository abstractions
```

while domain-specific concepts such as:

EXAMPLE / FLOW

```
Gift processing

Allocation rules

Tenant-aware Gift queries

Campaign lifecycle
```

may still deserve specific types and services instead of being forced into generic infrastructure.

Next roadmap topic: Topic 21 — Collections.